知识库管理
前后端对接与数据

数据分四处存放：管理状态在 MySQL（你的两张表），原文件在文件存储，切片和向量在 Azure 索引，关系在 Neo4j。你的代码只读写前两处。后两处只能通过同事的 SDK 写入，你只拿回一个"切片数"。前端看到的所有数据，都来自 MySQL 这两张表。

I. 数据在哪、谁负责

1. 四处数据

核心

存在哪	存什么	谁写	谁读
MySQL `library_kb_document`	每个文档一行：文件名、类型、大小、内容哈希、状态、切片数、最后的错误、来源	你的接口和 worker	前端（通过接口）
MySQL `library_kb_job`	每次处理一行：第几次尝试、心跳、错误码、开始和结束时间、这次用的文件	你的接口和 worker	worker；详情接口展示最近 10 条
文件存储（和 `/tap/ai/files` 是同一套）	原文件，路径为 `kb/{saas_id}/{job_id}.{ext}`	上传接口	worker 读出来交给 SDK
Azure 索引	切片、向量，供问答检索	只有 SDK	问答链路；同步接口只读清单
Neo4j	知识图谱关系	只有 SDK（开关打开时）	图谱功能

两张表由 `sql/003_create_library_kb_tables.sql` 创建，各环境的负责人去执行。公共字段 create_time、update_time、deleted、delete_time 照抄现有表，时间都是毫秒时间戳。

2. 三个关键编号

别搞混

编号	格式	用途
`id`	数据库主键	前端调详情、重试时传的 document_id 就是它
`document_key`	`doc::` 加 16 位十六进制	传给 SDK，作为索引和图谱里的文档编号。**文件更新后不变**，SDK 会按它删掉旧切片、写入新切片
`content_sha256`	文件内容的哈希	判断"内容有没有变"。从索引同步过来的文档没有原文件，这里为空字符串

同一个租户（saas_id）下，文件名和 document_key 都唯一。文件名的比较方式是：去掉路径、统一全角和半角、去掉首尾空格、转小写，所以 `A.pdf` 和 `a.pdf` 算同一个文件。

II. 和后端同事对接

1. 两个插座：你和 SDK 之间的全部数据

K9

入库：`ingest_file(saas_id, document_key, file_name, content)`

- 传进去：租户、固定的文档编号、文件名、文件内容（字节）。
- 拿回来：是否成功、切片数、错误码、错误信息。不会抛出异常，SDK 报错也转成"失败"返回。
- 拿回来的切片数写进文档表的 chunk_count，统计接口的切片总数就从这里来。

索引清单：`list_documents(saas_id)`，只读

- 拿回来：索引里每个文档的 document_key、文件名、切片数。
- 同步接口用它把"以前直接写进索引、但文档表里没有记录"的文档补登记进来。

2. 要跟同事确认的数据问题

明天问

- **租户和索引的对应关系**：不同 saas_id 写到哪个索引？有没有现成的映射可以复用？
- **更新文件时的旧数据**：同一个 document_key 重新入库，Azure 的旧切片会删掉（SDK 里已确认）；Neo4j 的旧关系会不会撤销？
- **部分成功**：Azure 写成功、Neo4j 写失败时，返回的状态是什么？我这边应该记为成功还是失败？
- **切片数**：返回结果里的切片数，是不是索引里这个文档最终的切片数？
- **索引清单**：按 document_id 分组统计，就能列出已有文档和切片数，这样可以吗？还是有现成的方法？
- **安装**：SDK 要求 Pillow<11、pypdf<5，后端是 Pillow>=11.1、pypdf>=5.1，两边装不到一起，能不能放宽版本？用私有包仓库还是 git 依赖？

3. 其他后端资源

- **存储**：不用改。已确认清理孤儿文件的任务只删附件表里登记过的文件，不会误删 `kb/` 下的文件。
- **密钥**：Azure 和 Neo4j 的密钥由同事负责，代码里不出现任何密钥。
- **权限**：由网关统一处理。
- **审计**：每次上传写一条 `library_kb.upload` 记录。
- **配置**：13 个 `TAP_AI_LIBRARY_KB_*` 环境变量，开关默认全部关闭；接上 SDK 以后才打开 worker。

III. 和前端对接

1. 6 个接口

/TAP/AI/LIBRARY/KB

接口	前端传	后端返回
`POST /stats`	租户字段	document_total、chunk_total、last_update_time、status_counts（每种状态的文档数）
`POST /documents/list`	keyword、status、file_ext、page、page_size	items、total、page、page_size，按更新时间倒序
`GET /documents/{document_id}`	文档的 id	文档信息，加最近 10 次处理记录
`POST /documents/upload`	multipart：多个 files，加租户字段	batch_id、queued、skipped、rejected 数量，加 items（每个文件的结果和原因）
`POST /documents/retry`	document_ids（1–100 个）	retried，加 ignored（每个没处理的文档及原因）
`POST /documents/sync`	租户字段	added、unchanged、conflicts、missing_in_index

租户字段是 saas_id、workspace_id、project_id、user_id，和 Library 其他接口一样。具体字段名最终以 K10 生成的接口文档为准。

2. 列表里每个文档的字段

页面怎么显示

字段	页面上怎么用
id	点详情、勾选重试时用
file_name、file_ext、size_bytes	文件名、类型、大小（字节，前端自行换算成 MB）
status	状态标签：pending 排队中、processing 处理中、indexed 已入库、failed 失败
chunk_count	切片数；只有 indexed 时有意义
last_error	失败原因；failed 时显示，重新排队后会清空
origin	来源：upload 为页面上传，index_sync 为从索引同步来的
indexed_time、update_time	毫秒时间戳

3. 上传后数据怎么变

判重规则

情况	结果	原因文字
文件名含 `..`、斜杠或控制字符	rejected	文件名不合法
类型不支持 / 空文件 / 超过 100MB	rejected	不支持的文件类型 / 文件为空 / 文件超过大小上限
同一批里文件名重复	rejected	同一批次中文件名重复
同名文件正在处理：内容相同 / 内容不同	skipped / rejected	已在处理中 / 该文件正在处理中，请稍后再上传
同名、内容相同、已入库	skipped	内容未变化
同名、内容相同、上次失败	queued	重新处理失败的文件
同名、内容不同	queued，文档编号不变	文件已更新
不同名、内容和已有文件相同（包括同一批里的）	skipped	与 xxx 内容相同
其他	queued，新建文档	新文件

入队的文件，文档状态变为 pending，列表里马上就能看到；处理完变为 indexed 或 failed。失败会自动重试，最多 3 次，之后才显示 failed。

4. 前端必须知道的规则

容易踩坑

- **返回格式**：成功时直接返回数据；业务错误也是 HTTP 200，内容为 `{code: "CCB001", message}`；参数不合法返回 422。
- **轮询**：列表里有 pending 或 processing 的文档时，定时刷新；全部变成 indexed 或 failed 后停止。
- **重试**：只对 failed 的文档有效。其他状态的文档、从索引同步来的文档（没有原文件），会出现在 ignored 里并带原因。
- **同步结果**：conflicts 是"文件名相同但文档编号不同"，missing_in_index 是"表里显示已入库，索引里却找不到"。这两种都只展示，不会自动修改数据。
- **限制**：支持 .txt .md .pdf .xlsx .xlsm .pptx，**不支持 .docx**；单个文件最大 100MB，一次最多 20 个。
- **本期不做删除。**

5. 提前说清楚

管理预期

SDK 接上之前，上传接口会返回"入库服务未配置"，同步接口会返回"索引清单服务未配置"。统计、列表、详情可以先联调，暂时是空数据。建议前端先按上面的字段，用假数据把状态标签、上传结果和轮询做好。

Made with Dia